

Task 3: Writing Basic SELECT Queries

Comprehensive Answers & Conceptual Foundations for Database Query Execution

Focus: Data Retrieval & Filtering

RDBMS: SQLite / MySQL Workbench

Scope: Core Syntax & Interview Prep

Interview Questions & Model Answers

Q1. What does SELECT * do?

`SELECT *` instructs the query engine to retrieve **all columns** across every record that satisfies the query conditions from the target table or view.

Production Caveat: While handy for rapid inspection during development, using `SELECT *` in production is an anti-pattern. It incurs needless I/O and network bandwidth, defeats covering index advantages, and can break calling code when table schemas change.

Q2. How do you filter rows?

Rows are filtered using the `WHERE` clause positioned directly after `FROM`. It evaluates predicate expressions against each row, keeping only rows that evaluate strictly to `TRUE` (ignoring `FALSE` and `NULL`).

```
SELECT emp_name, salary FROM employees WHERE department = 'Engineering';
```

Q3. What is LIKE '%value%'?

The `LIKE` operator performs fuzzy string pattern matching. The wildcard character `%` matches **zero or more characters**. Therefore, `'%value%'` filters for records where `'value'` exists anywhere within the target text (beginning, middle, or end).

Note: A leading wildcard like `'%. . .'` typically forces a full table scan because standard B-tree index lookups cannot leverage the column prefix.

Q4. What is BETWEEN used for?

`BETWEEN` checks if a value falls within a specified range. In SQL, `BETWEEN` is **inclusive** of both endpoints. `salary BETWEEN 60000 AND 90000` is logically equivalent to `(salary >= 60000 AND salary <= 90000)`. It works across numeric, date, and timestamp fields.

Q5. How do you limit output rows?

In SQLite, MySQL, and PostgreSQL, row restriction is implemented via the `LIMIT` clause at the end of the query: `LIMIT <n>` (optionally with `OFFSET` for pagination).

In ANSI SQL:2008 and newer database engines (Oracle 12c+, SQL Server 2012+), you can also use `OFFSET 0 ROWS FETCH FIRST <n> ROWS ONLY`.

Q6. What is the difference between = and IN?

- `=` is an exact scalar comparison operator that compares a column with exactly one discrete value (e.g., `role = 'Analyst'`).
- `IN` evaluates membership within a set or discrete list of values (e.g., `dept IN ('HR', 'Finance', 'Ops')`), behaving as shorthand for multiple chained `OR` conditions. It also accepts subqueries.

Q7. How to sort in descending order?

Append the `DESC` keyword after the relevant column reference in your `ORDER BY` clause:

```
SELECT employee_id, hire_date, salary FROM employees ORDER BY salary DESC;
```

Q8. What is aliasing?

Aliasing creates a temporary alias identifier for a column or table using the optional `AS` keyword. It is active only for the lifecycle of that specific query execution.

- **Column Aliasing:** Renames derived fields or aggregated outputs into human-readable headers (e.g., `SELECT COUNT(*) AS total_staff`).
- **Table Aliasing:** Simplifies verbose table names and is vital in self-joins or multiple-table relational joins (e.g., `FROM employee_records AS emp`).

Q9. Explain DISTINCT.

The `DISTINCT` modifier removes duplicate rows from the final result set. When applied to multiple columns, uniqueness is evaluated across the combined tuple of all selected columns rather than just the first field.

```
SELECT DISTINCT department, job_title FROM employees;
```

Q10. What is the default sort order?

The default sort order in SQL is **Ascending** (`ASC`). If you specify `ORDER BY column_name` without indicating `ASC` or `DESC`, the query automatically arranges values from smallest to largest (A to Z, earliest dates to latest dates, and lowest numeric values upward).